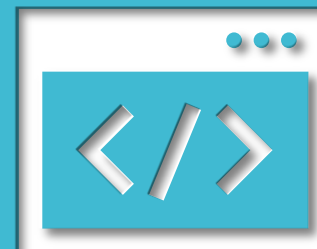


STRUCTURES DE DONNÉES & TYPES ABSTRAITS DE DONNÉES

PARTIE 1 : Files & Piles



1010
1010

INTRODUCTION

Vocabulaire & Concepts

INTRODUCTION

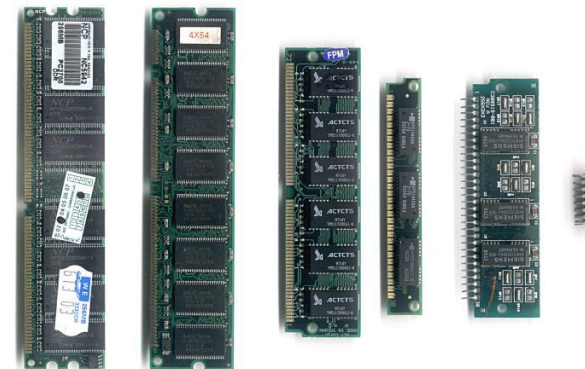
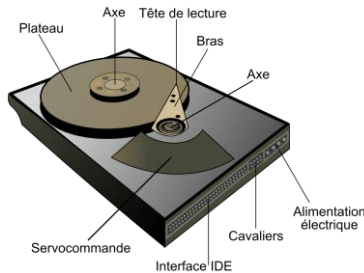
- **Rappels :**

- L'informatique est le **traitement automatisé de données** (numérisées).
- Les données et les programmes de traitement sont mis en **mémoire** de la machine.

INTRODUCTION

- La mémoire de nos ordinateurs :

- Nous limitons naturellement aux mémoires électroniques, elles existent sous de nombreux supports (solutions technologiques).
- Le choix d'une mémoire repose sur des critères : temps d'accès, volatilité, quantité, accès en lecture simple ou en écriture/lecteur, etc.



INTRODUCTION

- La mémoire de nos ordinateurs :

- Peut importe la technologie utilisée, l'usage général des mémoire repose la disposition suivante :
 - L'ensemble de **l'espace mémoire proposé est segmenté** en sous unités (zones/cellules/blocs, etc.) de même taille (= nombre de bits par cellule mémoire).
 - Chaque cellule est accessible (lecture et/ou écriture) par une adresse unique.
 - Les adresses mémoire sont des entiers.
 - Les valeurs mémorisées dans sont appelés également « mots » (une donnée codée étant constituée d'un ou plusieurs mots).

Adresse Mémoire	0x00 01	0x00 02	0x...
Valeurs / mots	1 001 111 100 000 000	0000 0000 1110 0001	...



INTRODUCTION

- Les **T.A.D.** (Types **A**bstracts de **D**onnées) :
 - Pour poursuivre notre travail sur les données/variables etc., nous allons les aborder comme étant des **T.A.D.**
 - Les **T.A.D.** sont descriptions des données et de leurs traitements (primitives) de façon totalement indépendantes de leur implémentation en mémoire de la machine.
 - Cela peut être vu comme un **cahier de charges** sur les données que l'on veut traiter, c'est-à-dire de consigner :
 - Ce que l'on veut mémoriser (nature des données)
 - Les **primitives** (traitements) que l'on veut appliquer sur les données).

INTRODUCTION

- **T.A.D. & Structures de Données :**

- Une structure de données est **la mise en œuvre/implémentation** en mémoire d'une TAD.
- Jusqu'à présent nous avons travaillé directement avec les structures de données proposée par Python :
 - **Variable de type entier**
 - **Variable de type flottant**
 - **Variable de type chaîne de caractères**
 - **Variable de type booléen**
 - **Collection séquentielle de type dictionnaire**
 - **Collection indexée de type liste Python**
 - **Les structures de type classe d'objet**

INTRODUCTION

- **Nos nouveaux objectifs :**

- **Objectif :** Implémenter de nouvelles structures de données adaptées à nos futurs besoins de traitement de données.

- **Pour ce faire nous procéderons en 2 étapes :**

1. Définir le nouveau TAD (Type Abstrait de Données) :
 - 1.a Définir les données traitées.
 - 1.b Définir l'ensemble des primitives associé au TAD
2. Proposer une implémentation du TAD.

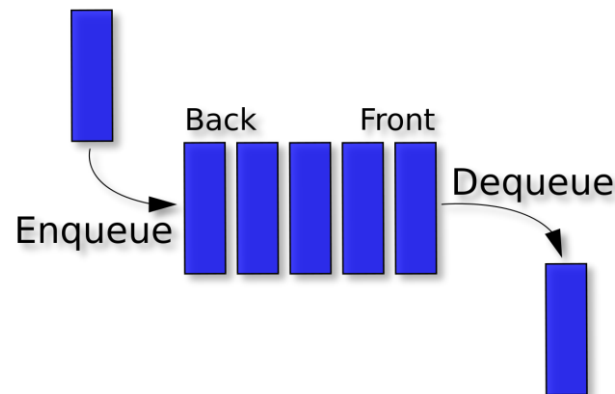
- **Éléments pour réaliser nos implémentations de TAD :**
 - La « coquille » de nos TAD sera implémentée par une plusieurs classes.
 - Les attributs et méthodes de nos classes seront implémentés à partir :
 - Des structures de données proposées par Python : variables, liste, etc.
 - De nos propres implémentation de structures de données préalablement réalisées.

LES FILES

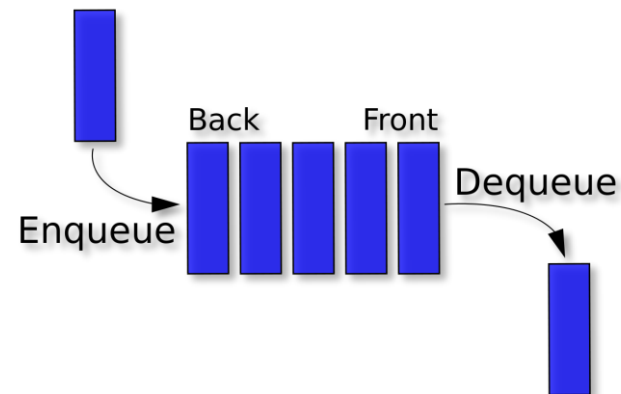
“First In, First Out”

- Description générale de la T.A.D. file :

- Une file est un type de donnée séquentielle où la première donnée entrée dans la file, sera la 1^{ère} à sortir. La 2^e à rentrer, la 2^e à sortir, etc.
- Les données sont traitées comme des personnes dans une file d'attente.
- On parle en anglaise de « **First In, First Out** » (**FIFO**) ou en français « **Premier Entré, Premier Sorti** » (**PEPS**)



- Description générale de la T.A.D. file :
 - En anglais on utilise le terme « **QUEUE** » pour désigner une file.



- **Exemples d'applications :**
 - Les mémoires tampons dans de nombreux sous systèmes.
 - Les files d'impression.
 - Algorithme de parcours en largeur (mémorisation des nœuds visiter).
 - En gestion des stocks.

- Primitives du T.A.D. file :

- **enfiler** (enqueue) : Rajouter un élément en fond de file (queue).
- **défiler** (dequeue) : Sortir l'élément en tête de file (queue).
- **vide** : Retourne true si la file est vide et false si la file n'est pas vide.
- **taille** : Retourne le nombre d'éléments contenu dans la file.

- Implémentation :

- Le constructeur instancie une liste **memFile** pour mémoriser les valeurs.
- Et un attribut **cpt** qui permet de tenir à jour le nombre d'éléments contenus dans la file.

```
class File:  
    def __init__(self):  
        self.memFile = []  
        self.cpt = 0
```

- Implémentation :

- Les primitives sont implémentées par 4 méthodes
- Les méthodes **vide** et **taille** ne posent pas de difficultés.

```
def vide(self):  
    if self.cpt == 0:  
        return True  
    else:  
        return False  
  
def taille(self):  
    return self.cpt
```


- Implémentation :

- La méthode **enfiler** va tout simplement rajouter la valeur à la liste **memFile**
- Il faut par contre bien veiller à incrémenter **cpt**

```
def enfiler(self, val):  
    self.memFile.append(val)  
    self.cpt += 1
```

- Implémentation :

- La méthode **defiler** va décrémenter le compteur **cpt**
- Elle va extraire la donnée de tête de file avec la méthode **pop** sur l'indice **0**.

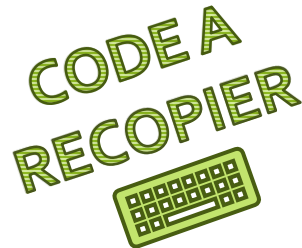
```
def defiler(self):  
    self.cpt -= 1  
    return self.memFile.pop(0)
```

- TRAVAIL A FAIRE :

- Implémenter la classe File avec le code proposé dans un 1^{er} temps :

```
5
6 class File:
7     def __init__(self):
8         self.memFile = []
9         self.cpt = 0
10
11     def vide(self):
12         if self.cpt == 0:
13             return True
14         else:
15             return False
16
17     def taille(self):
18         return self.cpt
19
```

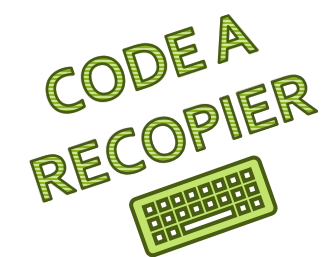
```
19
20     def enfiler(self, val):
21         self.memFile.append(val)
22         self.cpt += 1
23
24     def defiler(self):
25         self.cpt -= 1
26         return self.memFile.pop(0)
27
```



• TRAVAIL A FAIRE :

- Procédons maintenant à un jeu de tests pour valider l'implémentation.
- Commençons par instancier un objet file.
- Vérifions que les méthodes **vide()** et **taille()** sont opérationnelles pour une file vide.

```
# Instance d'un objet file :  
print("Instance d'une file \"fileTest\"")  
fileTest = File()  
  
# Vérification méthode vide :  
print("Le statut vide de la file est :", fileTest.vide())  
  
# Vérification de la taille :  
print("La file contient :", fileTest.taille(), "élément(s)).")
```



- TRAVAIL A FAIRE :

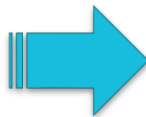
- Procédons à l'ajout de valeurs et vérifions les valeurs en affichant directement le contenu de la liste **memFile**

```
N = 10

# Peuplement avec une série numérique croissante :
for i in range(N):
    fileTest.enfiler(i)

print("Contenu de la liste :", fileTest.memFile)
```

CODE A
RECOPIER



```
Instance d'une file "fileTest"
Le statut vide de la file est : True
La file contient : 0 élément(s).
Contenu de la liste : [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

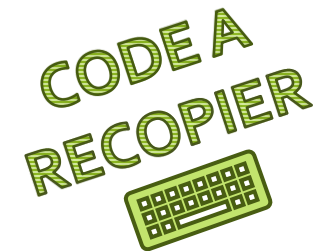
• TRAVAIL A FAIRE :

- Validons maintenant les méthodes **vide()** et **taille()** sont opérationnelles pour une file NON vide.

```
# Vérification méthode vide :  
print("Le statut vide de la file est :", fileTest.vide())  
  
# Vérification de la taille :  
print("La file contient :", fileTest.taille(), "élément(s)).")
```



```
Le statut vide de la file est : False  
La file contient : 10 élément(s)).
```



- TRAVAIL A FAIRE :

- Il nous reste maintenant à valider la méthode **defiler()**
- Appliquer le test suivant :

```
# Vérification de l'ordre PEPS/FIFO :
erreur = False
for i in range(N):
    if i != fileTest.defiler():
        erreur = True
if erreur:
    print("Erreur sur les données restituées !")
else:
    print("L'ensemble des données sont restituées selon l'ordre PEPS.")
```



L'ensemble des données sont restituées selon l'ordre PEPS.

- TRAVAIL A FAIRE :

- Vous avez implémentez grâce à des méthodes les 4 principales primitives de la T.AD. file.
- Vous allez maintenant rajouter une nouvelle méthode de type `__str__(self)` qui va nous permettre de réaliser d'afficher simplement le contenu de la file en passant l'identifiant de la file comme argument de la fonction `print()`
- **Implémentez cette méthode.**

`print(fileTest)` ➡ `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

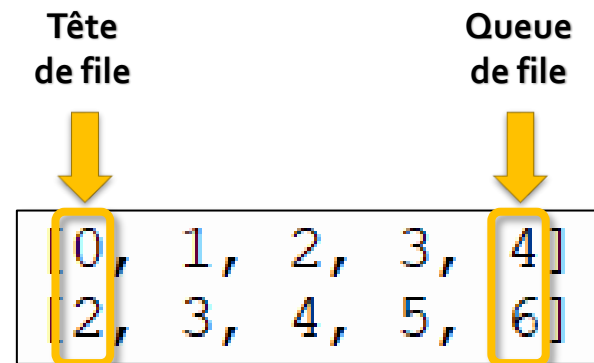
• TRAVAIL A FAIRE :

- Ultime évolution pour notre file, on vous propose maintenant de créer une nouvelle classe que l'on nommera **Tampon**.
- Cette classe va « exploiter » en grande partie notre classe **File** pour se définir.
- En effet la classe méthode intègre l'ensemble des méthodes de **File**. Mais on va modifier le comportement de la méthode **enfiler()** et rajouter un argument **tailleMaxi** au constructeur.
- Ainsi un objet de type Tampon se comportera comme un objet **File** tant que le nombre d'éléments stockés ne dépasse **tailleMaxi**.
- Si on ajoute un élément alors que l'objet Tampon est saturé. L'élément mémorisé en tête de file est effacé pour libérer de l'espace et la nouvelle valeur est rajouté en queue de file.

- TRAVAIL A FAIRE :

- Voici un exemple du comportement attendu :

```
# Instanciation d'un objet Tampon :  
tamponTest = Tampon(5)  
  
# Ajout des valeurs de test :  
for i in range(5):  
    tamponTest.enfiler(i)  
  
print(tamponTest)  
  
tamponTest.enfiler(5)  
tamponTest.enfiler(6)  
  
print(tamponTest)
```



• TRAVAIL A FAIRE :

- Compléter le code proposé et testez-le.



POUR ALLER PLUS LOIN

On aurait pu obtenir un résultat très proche en utilisant le concept d'héritage de la POO. Mais cela est hors programme, mais dans le principe, la classe Tampon pour récupérer les méthodes de la classe File en rédigeant son entête de cette manière :

```
class Tampon(File):
```

```
class Tampon:
    def __init__(self, tailleMaxi = 100):
        self.fileInterne = File()
        self.tMax = tailleMaxi

    def enfiler(self, val):

    def defiler(self):
        return self.fileInterne.defiler()

    def taille(self):
        return self.fileInterne.taille()

    def vide(self):
        return self.fileInterne.vide()

    def __str__(self):
        return str(self.fileInterne)
```

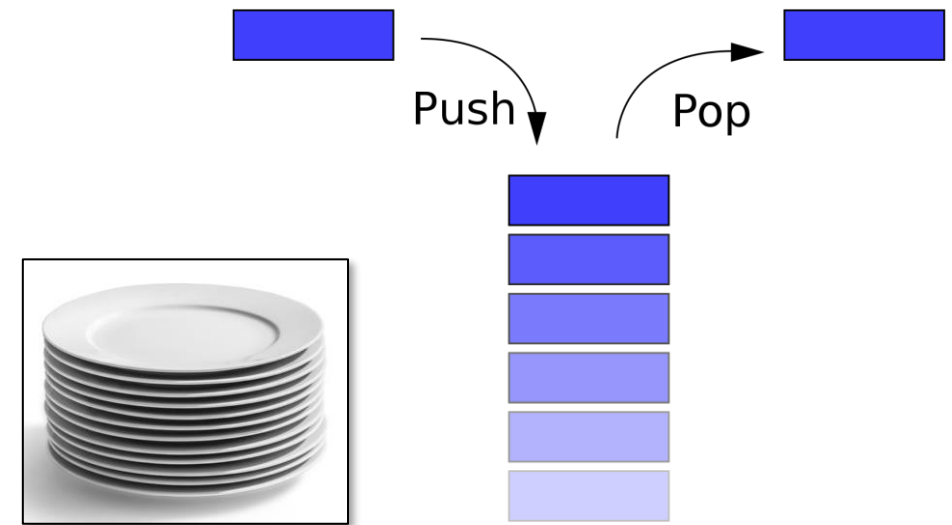
Compléter le code

LES PILES

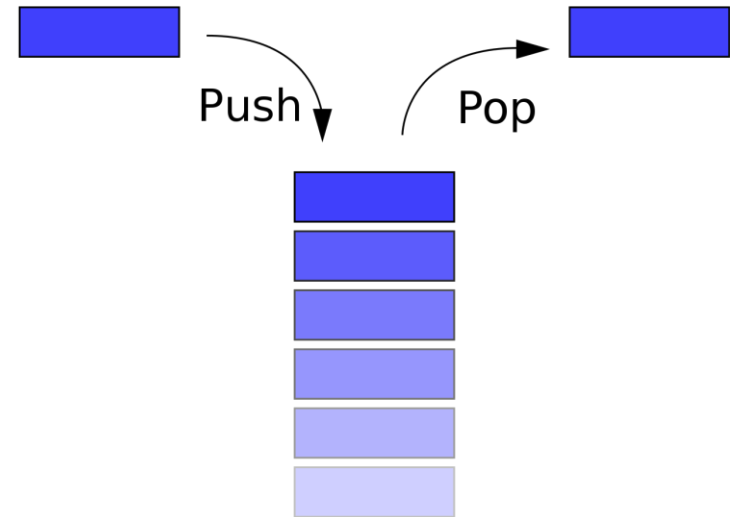
“Last In, First Out”

- Description générale de la T.A.D. pile :

- Une file est un type de donnée séquentielle où la dernière donnée entrée dans la pile, sera la 1^{ère} à sortir.
- Les données sont traitées comme des assiettes empilées.
- On parle en anglaise de « **Last In, First Out** » (**LIFO**) soit en français « Dernier Entré, Premier Sorti ».



- Description générale de la T.A.D. pile :
 - En anglais on utilise le terme « **STACK** » pour désigner une pile.



- **Exemples d'applications :**

- Cette structure est tout à fait intéressante quand on cherche à inverser un ordre des données et/ou donner priorité au dernier élément, voici quelques exemples pratiques :

- La fonction « undo/annuler» des logiciels 
- La pile d'appels récursifs quand vous utilisez des fonctions récursives.
- Application d'un algorithme de parcours en profondeur.
- Historique de navigation d'un navigateur web.

- Primitives du T.A.D. pile :
 - **empiler** (push) : **Rajouter** un élément en tête de pile (stack).
 - **dépiler** (pop) : **Sortir** l'élément en tête de pile (stack).
 - **vide** : Retourne **true** si la pile est vide et **false** si la pile n'est pas vide.
 - **taille** : Retourne le **nombre d'éléments** contenus dans la pile.

- Implémentation :

- On procède comme avec le T.A.D. de la file, on implémente une classe **Pile**.
- Le constructeur instancie une liste **memFile** pour mémoriser les valeurs.
- Et un attribut **cpt** qui permet de tenir à jour le nombre d'éléments contenus dans la file.

```
class Pile:  
    def __init__(self):  
        self.memFile = []    # On implémente le stockage pas une liste.  
        self.cpt = 0        # Compteur du nombre d'éléments contenus dans la pile.
```

- **Implémentation :**

- Le jeu de test reste identique à 90%. La différence réside sur la partie du code qui assure la vérification de la chronologie LIFO.



Suite du code page
suivante

```
39 # Instance d'un objet file :
40 print("Instance d'une pile \"pileTest\"")
41 pileTest = Pile()
42
43 # Vérification méthode vide :
44 print("Le statut vide de la pile est :", pileTest.vide())
45
46 # Vérification de la taille :
47 print("La pile contient :", pileTest.taille(), "élément(s)).")
48
49 N = 10
50 # Peuplement avec une série numérique croissante :
51 for i in range(N):
52     pileTest.empiler(i)
53
54 print("Contenu de la liste :", pileTest.memFile)
55 print(pileTest)
56
57 # Vérification méthode vide :
58 print("Le statut vide de la file est :", pileTest.vide())
59
60 # Vérification de la taille :
61 print("La file contient :", pileTest.taille(), "élément(s)).")
62
```

- Implémentation :

- Le jeu de test reste identique à 90%. La différence réside sur la partie du code qui assure la vérification de la chronologie LIFO.

```
63 # Vérification de l'ordre LIFO :
64 erreur = False
65 for i in range(N-1, -1, -1):
66     if i != pileTest.depiler():
67         erreur = True
68 if erreur:
69     print("Erreur sur les données restituées !")
70 else:
71     print("L'ensemble des données sont restituées selon l'ordre LIFO.")
72
```

- **Travail à faire :**

- Implémenter toute la classe Pile.
- Tester votre code avec le jeu de tests proposé précédemment.



FIN